TD — gRPC : Python / Go

Objectif :

Faire communiquer deux applications écrites dans des langages différents grâce à gRPC, en partageant un seul contrat Protocol Buffers (`.proto`).

Prérequis

- Python 3.14, `pip`
- Go 1.25
- le compilateur Protocol Buffers (`protoc`).

Remarque sur la sécurité

Les exemples utilisent `insecure_channel` côté Python et `insecure.NewCredentials()` côté Go pour simplifier les tests locaux.

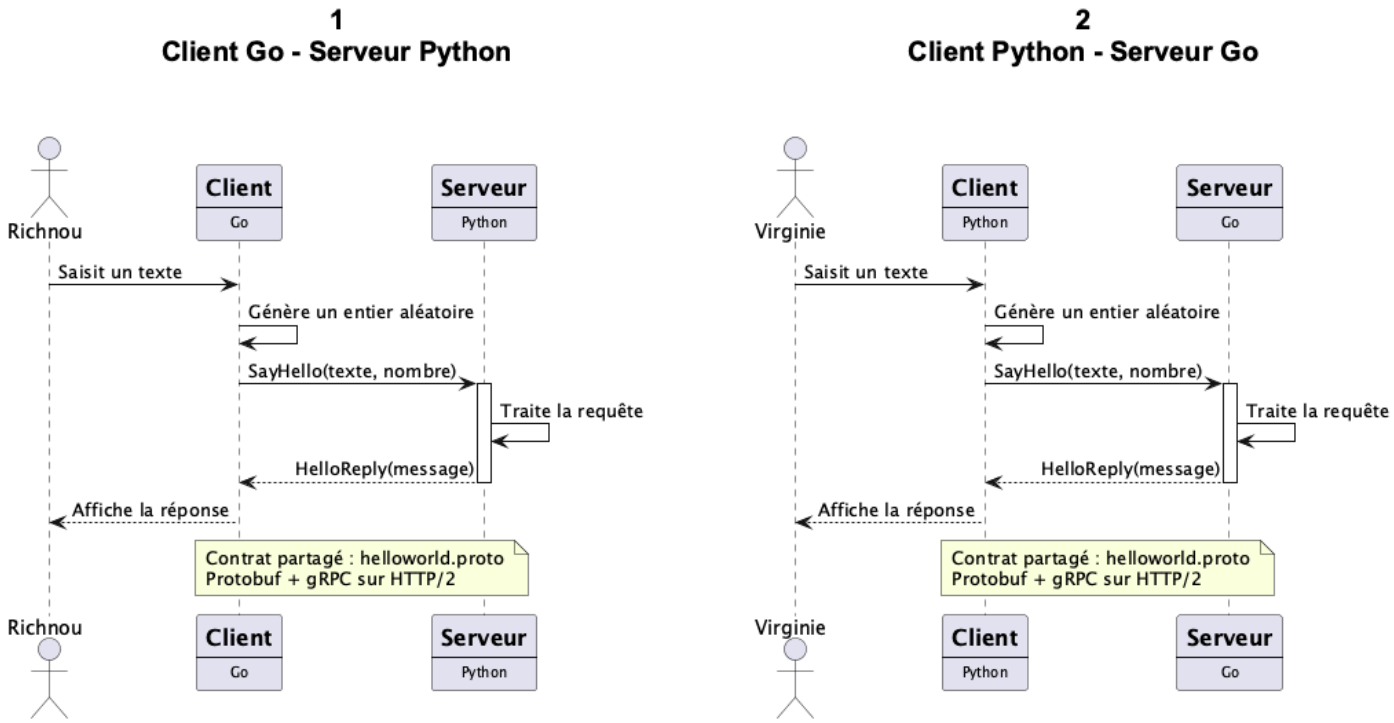
Ces connexions ne chiffrent pas les échanges et ne doivent pas être utilisées telles quelles sur un réseau non fiable.

1. Ce que l'on va construire

On définit un service `Greeter` avec une méthode RPC `SayHello`.

Le client demande à l'utilisateur de saisir un texte au clavier et génère un entier aléatoire. Il transmet ces deux valeurs au serveur, qui renvoie une réponse reprenant les informations reçues.

Les deux applications utilisent exactement le même fichier `helloworld.proto`.



Note : on peut envisager Client Go / Serveur Go, idem Python...

2. Préparer l'arborescence

Depuis un répertoire de travail :

```
mkdir -p grpc-td/proto
mkdir -p grpc-td/python
mkdir -p grpc-td/go/server
mkdir -p grpc-td/go/client
cd grpc-td
```

On obtiendra cette structure

```
/
├── proto/
│   └── helloworld.proto
├── python/
│   ├── server.py
│   └── client.py
└── go/
    ├── go.mod
    ├── server/
    │   └── main.go
    └── client/
        └── main.go
```

3. Écrire le contrat `.proto`

```
touch proto/helloworld.proto
```

```
syntax = "proto3";

package helloworld;

option go_package = "example.com/grpc-td/pb;pb";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest {
  string texte = 1;
  int32 nombre = 2;
}

message HelloReply {
  string message = 1;
}
```

4. Installer les outils et générer le code

Il faut le compilateur `protoc`, puis les greffons de génération propres à chaque langage.

Installer `protoc`

Installez le compilateur Protocol Buffers en suivant la documentation officielle :

https://protobuf.dev/installation/

```
protoc --version
```

Générer les fichiers Python

Créez et activez un environnement virtuel Python :

```
python -m venv venv

source venv/bin/activate    # Linux / macOS
venv\Scripts\Activate.ps1   # Windows PowerShell

pip install --upgrade grpcio grpcio-tools
```

Générez les **stubs** Python depuis la racine du projet :

```
python -m grpc_tools.protoc \
  -I proto \
  --python_out=python \
  --grpc_python_out=python \
  proto/helloworld.proto
```

Deux fichiers sont générés dans `python/` :

- `helloworld_pb2.py` : classes des messages Protobuf ;
- `helloworld_pb2_grpc.py` : stubs client et base de serveur gRPC.

Générer les fichiers Go

Initialisez le module Go dans le répertoire `go/` :

```
cd go
go mod init example.com/grpc-td
```

Linux / macOS :

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

# Ajouter dans le ~/.bashrc pour modif permanente
export PATH="$(go env GOPATH)/bin:$PATH"
```

Windows PowerShell :

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

# Ajouter aux variables d'env pour modif permantence
$env:Path = "$(go env GOPATH)\bin;$env:Path"
```

Tester

```
protoc-gen-go --version
protoc-gen-go-grpc --version
protoc --version
```

Générer les proto

```
protoc -I ../proto \
  --go_out=. \
  --go_opt=module=example.com/grpc-td \
  --go-grpc_out=. \
  --go-grpc_opt=module=example.com/grpc-td \
  ../proto/helloworld.proto
```

Les fichiers générés se trouvent dans `go/pb/` :

- `helloworld.pb.go` : messages Protobuf ;
- `helloworld_grpc.pb.go` : interfaces et code gRPC.

Récupérez les dépendances du module :

```
go mod tidy
```

Les dépendances Go sont gérées par `go.mod` et `go.sum` . Aucune configuration manuelle de GOPATH n'est nécessaire.

5. Écrire le serveur Python

Créez `python/server.py` :

```python
# python/server.py
from concurrent import futures
import grpc
import helloworld_pb2
import helloworld_pb2_grpc

class Greeter(helloworld_pb2_grpc.GreeterServicer):
    def SayHello(self, request, context):
        print(
            f"Requête reçue : texte={request.texte!r}, "
            f"nombre={request.nombre}"
        )
        return helloworld_pb2.HelloReply(
            message=(
                f"J'ai reçu le texte « {request.texte} » "
                f"et le nombre {request.nombre}. "
                f"(réponse Python)"
            )
        )

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    helloworld_pb2_grpc.add_GreeterServicer_to_server(
        Greeter(), server
    )
    server.add_insecure_port("[::]:50051")
    server.start()
    print("Serveur Python démarré sur le port 50051")
    server.wait_for_termination()

if __name__ == "__main__":
    try:
        serve()
    except KeyboardInterrupt:
        print("\nArrêt du serveur.")
```

6. Écrire le client Go

Créez `go/client/main.go` :

```go
// main.go
package main

import (
    "bufio"
    "context"
    "fmt"
    "log"
    "math/rand/v2"
    "os"
    "strings"
    "time"

    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"

    pb "example.com/grpc-td/pb"
)

func main() {
    reader := bufio.NewReader(os.Stdin)

    fmt.Print("Saisissez un texte : ")
    texte, err := reader.ReadString('\n')
    if err != nil && len(texte) == 0 {
        log.Fatalf("lecture du texte impossible : %v", err)
    }
    texte = strings.TrimSpace(texte)

    nombre := rand.IntN(1000)
    fmt.Printf("Nombre aléatoire généré : %d\n", nombre)

    conn, err := grpc.NewClient(
        "localhost:50051",
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )
    if err != nil {
        log.Fatalf("création du client gRPC impossible : %v", err)
    }
    defer conn.Close()

    client := pb.NewGreeterClient(conn)
```

```
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    response, err := client.SayHello(ctx, &pb.HelloRequest{
        Texte:  texte,
        Nombre: int32(nombre),
    })
    if err != nil {
        log.Fatalf("appel RPC en échec : %v", err)
    }

    fmt.Printf("Réponse du serveur : %s\n", response.GetMessage())
}
```

7. Tester Python serveur + Go client

Ouvrez deux terminaux depuis la racine du projet.

Terminal A — serveur Python :

```
source venv/bin/activate
python python/server.py
```

Sous Windows PowerShell, activez l'environnement virtuel avec `venv\Scripts\Activate.ps1` .

Terminal B — client Go :

```
cd go
go run ./client
```

8. Écrire le serveur Go

Créez `go/server/main.go` :

```go
// go/server/main.go
package main

import (
    "context"
    "fmt"
    "log"
    "net"

    "google.golang.org/grpc"

    pb "example.com/grpc-td/pb"
)

type greeterServer struct {
    pb.UnimplementedGreeterServer
}

func (s *greeterServer) SayHello(
    ctx context.Context,
    req *pb.HelloRequest,
) (*pb.HelloReply, error) {
    log.Printf(
        "Requête reçue : texte=%q, nombre=%d",
        req.GetTexte(),
        req.GetNombre(),
    )

    return &pb.HelloReply{
        Message: fmt.Sprintf(
            "J'ai reçu le texte « %s » et le nombre %d. (réponse Go)",
            req.GetTexte(),
            req.GetNombre(),
        ),
    }, nil
}

func main() {
    listener, err := net.Listen("tcp", ":50051")
    if err != nil {
        log.Fatalf("écoute impossible : %v", err)
    }
```

```
    server := grpc.NewServer()
    pb.RegisterGreeterServer(server, &greeterServer{})

    log.Println("Serveur Go démarré sur le port 50051")
    if err := server.Serve(listener); err != nil {
        log.Fatalf("serveur arrêté : %v", err)
    }
}
```

9. Écrire le client Python

Créez `python/client.py` :

```python
# python/client.py
import random

import grpc

import helloworld_pb2
import helloworld_pb2_grpc


def main():
    texte = input("Saisissez un texte : ")
    nombre = random.randint(0, 999)

    print(f"Nombre aléatoire généré : {nombre}")

    with grpc.insecure_channel("localhost:50051") as channel:
        client = helloworld_pb2_grpc.GreeterStub(channel)

        response = client.SayHello(
            helloworld_pb2.HelloRequest(
                texte=texte,
                nombre=nombre,
            ),
            timeout=5,
        )

        print(f"Réponse du serveur : {response.message}")


if __name__ == "__main__":
    main()
```

10. Tester Go serveur + Python client

Terminal A — serveur Go :

```
cd go
go run ./server
```

Terminal B — client Python :

```
source venv/bin/activate
python python/client.py
```

11. On mélange ?

On peut aussi essayer :

```
source venv/bin/activate
python python/server.py
python python/client.py
```

ou

```
cd go
go run ./server
go run ./client
```